

# Small Language Models

GUIA DIDÁTICO SOBRE SMALL LANGUAGE MODELS (SLMs) E  
FUNDAMENTOS DE LLMS

BASEADO NOS ARTIGOS CIENTÍFICOS DO ESTADO DA ARTE: GEMMA 4, LFM2,  
MINISTRAL 3, PHI-4 E PESQUISAS DE FRONTEIRA

Beatriz Almeida

11 de setembro de 2026

# Índice

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Representação de Texto e Fundamentos de Embeddings</b>         | <b>3</b>  |
| 1.1      | Tokenização . . . . .                                             | 3         |
| 1.1.1    | Conceito . . . . .                                                | 3         |
| 1.1.2    | Matemática e o Algoritmo BPE (Byte-Pair Encoding) . . . . .       | 3         |
| 1.2      | Embeddings . . . . .                                              | 5         |
| 1.2.1    | Conceito . . . . .                                                | 5         |
| 1.2.2    | Matemática: Matrizes de Embedding Entrada e Saída . . . . .       | 5         |
| 1.3      | Exercícios e Resumos . . . . .                                    | 6         |
| 1.3.1    | Resumo do Capítulo . . . . .                                      | 6         |
| 1.3.2    | Exercícios Resolvidos . . . . .                                   | 6         |
| <b>2</b> | <b>Arquitetura Transformer e Mecanismos de Atenção</b>            | <b>8</b>  |
| 2.1      | Arquitetura Completa e o Fluxo Sequencial . . . . .               | 8         |
| 2.2      | Self-Attention (Autoatenção) . . . . .                            | 8         |
| 2.2.1    | Conceito . . . . .                                                | 8         |
| 2.2.2    | Matemática: Fórmula Clássica . . . . .                            | 8         |
| 2.2.3    | Exemplo Numérico . . . . .                                        | 9         |
| 2.3      | Multi-Head, MQA e Grouped-Query Attention (GQA) . . . . .         | 10        |
| 2.4      | Positional Encoding e Rotary Position Embeddings (RoPE) . . . . . | 10        |
| 2.4.1    | Conceito . . . . .                                                | 10        |
| 2.4.2    | Matemática do RoPE (Rotary Position Embedding) . . . . .          | 10        |
| 2.5      | RMSNorm e SwiGLU . . . . .                                        | 11        |
| 2.5.1    | RMSNorm (Root Mean Square Layer Normalization) . . . . .          | 11        |
| 2.5.2    | SwiGLU (Swish Gated Linear Unit) . . . . .                        | 11        |
| 2.6      | Exercícios e Resumos . . . . .                                    | 12        |
| 2.6.1    | Resumo do Capítulo . . . . .                                      | 12        |
| 2.6.2    | Exercícios . . . . .                                              | 12        |
| <b>3</b> | <b>Funcionamento Extremo, Memória e Ciclo de Pré-treinamento</b>  | <b>13</b> |
| 3.1      | Arquitetura Completa de uma LLM: O Fluxo de Dados . . . . .       | 13        |
| 3.2      | Parâmetros, Precisão e Pegada de Memória . . . . .                | 13        |
| 3.2.1    | Diferenciando Parâmetros Totais de Parâmetros Ativados . . . . .  | 13        |
| 3.2.2    | Precisão Numérica e Memória de Armazenamento . . . . .            | 14        |
| 3.3      | Ciclo de Pré-treinamento . . . . .                                | 14        |
| 3.3.1    | Loss de Entropia Cruzada Causal . . . . .                         | 15        |
| 3.3.2    | AdamW (Decoupled Weight Decay Adam) . . . . .                     | 15        |
| 3.4      | Exercícios e Resumos . . . . .                                    | 15        |
| 3.4.1    | Resumo do Capítulo . . . . .                                      | 15        |
| 3.4.2    | Exercícios . . . . .                                              | 16        |

|          |                                                                                  |           |
|----------|----------------------------------------------------------------------------------|-----------|
| <b>4</b> | <b>Pós-treinamento, Alinhamento de Preferências e Raciocínio</b>                 | <b>17</b> |
| 4.1      | Pré-treinamento vs. Fine-tuning vs. Pós-treinamento . . . . .                    | 17        |
| 4.2      | Instruction Tuning e SFT . . . . .                                               | 17        |
| 4.3      | RL Aplicado a LLMs: PPO, DPO e GRPO . . . . .                                    | 17        |
| 4.4      | Chain-of-Thought (CoT) vs. Raciocínio (Reasoning) . . . . .                      | 18        |
| 4.5      | Exercícios e Resumos . . . . .                                                   | 18        |
| 4.5.1    | Resumo do Capítulo . . . . .                                                     | 19        |
| 4.5.2    | Exercícios . . . . .                                                             | 19        |
| <b>5</b> | <b>Análise Crítica e Comparativa dos Artigos Científicos</b>                     | <b>20</b> |
| 5.1      | Gemma 4: Fronteira de Multimodalidade e Raciocínio (DeepMind) . . . . .          | 20        |
| 5.1.1    | Problema e Motivação . . . . .                                                   | 20        |
| 5.1.2    | Arquitetura e Inovações Técnicas . . . . .                                       | 20        |
| 5.1.3    | Resultados e Benchmarks . . . . .                                                | 20        |
| 5.2      | LFM2: Liquid Foundation Models de Extrema Velocidade (Liquid AI) . . . . .       | 20        |
| 5.2.1    | Problema e Motivação . . . . .                                                   | 20        |
| 5.2.2    | Arquitetura: Minimal Hybrid . . . . .                                            | 21        |
| 5.2.3    | Pre-training: Decoupled Top-K Distillation . . . . .                             | 21        |
| 5.2.4    | Modelos Especializados e Resultados . . . . .                                    | 21        |
| 5.3      | Minstral 3: Cascade Distillation e Análise de Professores (Mistral AI) . . . . . | 21        |
| 5.3.1    | Problema e Motivação . . . . .                                                   | 21        |
| 5.3.2    | Arquitetura e Cascade Distillation . . . . .                                     | 21        |
| 5.3.3    | Ablações Técnicas Cruciais do Artigo . . . . .                                   | 21        |
| 5.4      | Phi-4: Supremacia Lógica via Dados Sintéticos Criteriosos (Microsoft) . . . . .  | 22        |
| 5.4.1    | Problema e Motivação . . . . .                                                   | 22        |
| 5.4.2    | Estratégia de Dados de Pretraining . . . . .                                     | 22        |
| 5.4.3    | Post-Training: Pivotal Token Search (PTS) DPO . . . . .                          | 22        |
| 5.4.4    | Métricas, Resultados e Limitações . . . . .                                      | 22        |
| 5.5      | Tabela de Síntese Comparativa . . . . .                                          | 23        |
| 5.6      | Exercícios e Resumos . . . . .                                                   | 23        |
| 5.6.1    | Resumo do Capítulo . . . . .                                                     | 23        |
| 5.6.2    | Exercícios . . . . .                                                             | 23        |

# Capítulo 1

## Representação de Texto e Fundamentos de Embeddings

Para compreender como modelos de linguagem processam informações, devemos primeiro entender como o texto bruto do mundo humano é transformado em estruturas numéricas que os computadores podem manipular. Esse capítulo cobre os dois primeiros passos desse pipeline: a tokenização e os embeddings.

### 1.1 Tokenização

#### 1.1.1 Conceito

A tokenização é o processo de segmentar uma string de texto bruto em unidades menores e discretas chamadas tokens. Esses tokens podem ser caracteres individuais, palavras inteiras ou, mais frequentemente no estado da arte atual, partes de palavras (subpalavras). O conjunto de todos os tokens únicos que um modelo pode reconhecer constitui o seu vocabulário ( $V$ ).

#### Intuição: Subpalavras e Tokens Especiais

Se mapearmos apenas palavras completas, nosso vocabulário seria infinitamente grande devido a neologismos, erros de digitação e variações morfológicas (ex: "correr", "corri", "correrão"). Por outro lado, mapear apenas caracteres individuais (ex: "c", "o", "r") faria com que o modelo processasse sequências longas demais e perdesse as relações semânticas complexas contidas nas palavras. A tokenização por subpalavras (subwords) encontra o equilíbrio: reconstrói palavras raras ou novas a partir de fragmentos comuns, enquanto mantém palavras frequentes intactas. Além dos tokens de texto, usamos tokens especiais para sinalizar estruturas específicas ao modelo (como fim de texto `<|endoftext|>`, quebra de turno de chat ou instruções).

#### 1.1.2 Matemática e o Algoritmo BPE (Byte-Pair Encoding)

O BPE é o algoritmo de tokenização por subpalavras mais utilizado. Ele começa tratando cada byte (ou caractere) individual como um token inicial. O algoritmo conta a frequência de todos os pares de tokens adjacentes em um corpus de treinamento e funde iterativamente o par mais frequente para formar um novo token. Esse processo se repete até que o vocabulário atinja o tamanho alvo determinado  $V = |V|$ .

A probabilidade ou frequência de fusão de um par  $(t_i, t_j)$  adjacente é simplesmente:

$$\text{Freq}(t_i, t_j) = \sum_{c \in C} I(c_k = t_i \wedge c_{k+1} = t_j)$$

onde  $C$  é o corpus e  $I$  é a função indicadora.

### Exemplo Prático: BPE Passo a Passo

Suponha que nosso corpus de treino contenha apenas três palavras com suas respectivas frequências:

- "b a n a n a"(frequência: 5)
- "a n a n a s"(frequência: 3)

Nossos caracteres/tokens iniciais (vocabulário inicial) são:  $\{a, b, n, s\}$ . Vamos fundir os pares mais frequentes passo a passo:

**1. Passo 1:** Contamos os pares de caracteres adjacentes no corpus:

- O par  $(a, n)$  aparece em "b a n a n a"(2 vezes por palavra  $\times 5 = 10$ ) e em "a n a n a s"(2 vezes por palavra  $\times 3 = 6$ ). Total de ocorrências = 16.
- O par  $(n, a)$  aparece em "b a n a n a"(2 vezes  $\times 5 = 10$ ) e em "a n a n a s"(1 vez  $\times 3 = 3$ ). Total = 13.
- O par  $(b, a)$  aparece em "b a n a n a"(1 vez  $\times 5 = 5$ ). Total = 5.
- O par  $(a, s)$  aparece em "a n a n a s"(1 vez  $\times 3 = 3$ ). Total = 3.

O par mais frequente é  $(a, n)$  com 16 ocorrências. Fundimos esse par no novo token "an". Nosso vocabulário passa a ser:  $\{a, b, n, s, an\}$ . O corpus atualizado fica:

- "b [an] a n a"  $\rightarrow$  "b [an] [an] a"(freq: 5)
- "[an] [an] a s"(freq: 3)

**2. Passo 2:** Recalculamos as frequências dos novos pares adjacentes:

- O par  $(an, an)$  aparece em "b [an] [an] a"(1 vez  $\times 5 = 5$ ) e em "[an] [an] a s"(1 vez  $\times 3 = 3$ ). Total = 8.
- O par  $(an, a)$  aparece em "b [an] [an] a"(1 vez  $\times 5 = 5$ ) e em "[an] [an] a s"(1 vez  $\times 3 = 3$ ). Total = 8.
- O par  $(b, an)$  aparece em "b [an] [an] a"(1 vez  $\times 5 = 5$ ). Total = 5.
- O par  $(a, s)$  aparece em "[an] [an] a s"(1 vez  $\times 3 = 3$ ). Total = 3.

Temos um empate entre  $(an, an)$  e  $(an, a)$ . Escolhemos fundir  $(an, a)$  no novo token "ana". Nosso vocabulário final passa a incluir "ana". Esse algoritmo constrói progressivamente tokens de subpalavras extremamente eficientes.

**Aplicação Prática nos Artigos: Tokenização nos Modelos Modernos**

Segundo os relatórios fornecidos:

- **Gemma 4** utiliza o SentencePiece com um vocabulário extremamente grande de 262k entradas, herdado do Gemini. Ele divide dígitos individuais e preserva espaços em branco usando encodificação em nível de byte (Byte-level) para evitar o problema de tokens desconhecidos.
- **Ministral 3** utiliza um vocabulário de 131k tokens, focado em eficiência computacional e suporte multilíngue otimizado.
- **LFM2** e seu modelo especializado em busca LFM2-ColBERT utilizam um tokenizador BPE em nível de byte com um vocabulário compacto de 65.536 tokens para maximizar a velocidade do processamento e reduzir a pegada de memória do cache de atenção nos modelos de menor escala (como o de 350M).

## 1.2 Embeddings

### 1.2.1 Conceito

Computadores não podem computar em cima de símbolos discretos (tokens). Precisamos converter cada token único  $t \in V$  em um vetor contínuo de alta dimensão em  $\mathbb{R}^{d_{\text{model}}}$ , onde  $d_{\text{model}}$  é a dimensão oculta do modelo. Essa representação vetorial captura as relações semânticas e sintáticas do token: tokens com significados parecidos são mapeados para posições próximas no espaço geométrico.

### 1.2.2 Matemática: Matrizes de Embedding Entrada e Saída

Temos duas matrizes de embeddings em uma LLM padrão decoder-only:

1. **Matriz de Embedding de Entrada** ( $W_{\text{in}} \in \mathbb{R}^{V \times d_{\text{model}}}$ ): Atua como uma tabela de busca (lookup). Dado o índice do token  $i \in \{1, \dots, V\}$ , a representação vetorial  $x \in \mathbb{R}^{d_{\text{model}}}$  é obtida extraindo a  $i$ -ésima linha de  $W_{\text{in}}$ .
2. **Matriz de Embedding de Saída / Unembedding** ( $W_{\text{out}} \in \mathbb{R}^{d_{\text{model}} \times V}$ ): Usada na última camada do modelo para converter a representação oculta final  $h \in \mathbb{R}^{d_{\text{model}}}$  de volta para pontuações brutas (logits) sobre todo o vocabulário:

$$\text{logits} = h \cdot W_{\text{out}}^T$$

#### Tied vs. Untied Embeddings (Embeddings Compartilhados)

Dizemos que os embeddings são tied (compartilhados) quando a mesma matriz física de parâmetros é utilizada para entrada e saída, ou seja,  $W_{\text{out}} = W_{\text{in}}$ . Se forem untied (desacoplados), as matrizes  $W_{\text{in}}$  e  $W_{\text{out}}$  são atualizadas de forma independente durante o treinamento.

**Intuição: Por que compartilhar Embeddings?**

Em modelos menores (SLMs), o tamanho do vocabulário  $V$  é muito grande em comparação com a dimensão interna do modelo  $d_{\text{model}}$ . Por exemplo, se  $V = 131.000$  e  $d_{\text{model}} = 3072$ , cada matriz de embedding contém  $131.000 \times 3072 \approx 402$  milhões de parâmetros. Ter duas matrizes distintas consumiria mais de 800 milhões de parâmetros apenas no processamento de entrada e saída. Compartilhar os embeddings poupa instantaneamente metade desses parâmetros, reduzindo o uso de memória RAM/VRAM. No entanto, desacoplar os embeddings (untied) dá mais capacidade de representação ao modelo: a matriz de entrada foca em capturar a semântica de leitura, enquanto a de saída foca na probabilidade de predição/escrita.

**Aplicação Prática nos Artigos: Estratégias de Embeddings nos Artigos**

- **Ministral 3 3B** utiliza embeddings tied (compartilhados) para otimizar sua estrutura de 3 bilhões de parâmetros, reduzindo custos de memória. Por outro lado, os modelos maiores da mesma família, Ministral 8B e 14B, utilizam embeddings untied para máxima fidelidade e capacidade de modelagem linguística.
- **Gemma 4 E2B e E4B** utilizam uma técnica ainda mais especializada: embeddings por camada (per-layer embeddings), permitindo que os embeddings de tamanho reduzido sejam reaproveitados ao longo da profundidade da rede para conservar parâmetros ativos e reduzir o uso de VRAM de forma dinâmica em dispositivos móveis.

### 1.3 Exercícios e Resumos

**Erro Conceitual Comum: Confundir Tokens com Sílabas**

Um erro clássico é assumir que o algoritmo de tokenização conhece regras gramaticais de divisão de sílabas. O BPE funciona puramente à base de estatísticas e frequências de caracteres e bytes. Ele não sabe o que é uma vogal ou uma consoante. Por exemplo, a palavra "computador" pode ser quebrada em ["comp", "ut", "ador"] puramente porque esses sub-padrões são frequentes em outras partes do corpus de treino.

#### 1.3.1 Resumo do Capítulo

- A tokenização reduz texto bruto a uma sequência de tokens numéricos em um vocabulário finito. O BPE constrói esse vocabulário fundindo iterativamente bytes frequentes.
- Os embeddings transformam tokens em vetores semânticos em um espaço geométrico  $\mathbb{R}^{d_{\text{model}}}$ .
- Tying de embeddings poupa centenas de milhões de parâmetros compartilhando pesos de entrada e saída, vital para SLMs de extrema eficiência no edge.

#### 1.3.2 Exercícios Resolvidos

**Exercício 1:** Considere uma LLM com vocabulário  $V = 100.000$  e dimensão interna  $d_{\text{model}} = 4096$ . Quantos parâmetros são economizados se utilizarmos embeddings compartilhados (tied) em vez de independentes (untied)?

**Solução:** Se os embeddings forem independentes (untied), precisamos de duas matrizes distintas:  $W_{\text{in}}$  e  $W_{\text{out}}$ . O total de parâmetros é:

$$2 \times (V \times d_{\text{model}}) = 2 \times (100.000 \times 4096) = 2 \times 409.600.000 = 819.200.000 \text{ parâmetros}$$

Ao compartilhar as matrizes (tied), usamos apenas uma matriz de tamanho  $V \times d_{\text{model}}$ . Portanto, economizamos exatamente:

$$1 \times (V \times d_{\text{model}}) = 409.600.000 \text{ parâmetros (aprox. 410 milhões de parâmetros)}$$

## Capítulo 2

# Arquitetura Transformer e Mecanismos de Atenção

Com os embeddings gerados, precisamos processar essa sequência de vetores para que cada token adquira contexto de seus vizinhos. O principal desse processamento é a arquitetura Transformer e, em especial, o mecanismo de Self-Attention.

### 2.1 Arquitetura Completa e o Fluxo Sequencial

A maioria das LLMs modernas e todos os artigos cobertos nesse guia utilizam a arquitetura Transformer Decoder-Only (apenas decodificador). Diferente do modelo clássico original (que tinha um codificador e um decodificador para tradução), o modelo decoder-only processa tokens autoregressivamente, ou seja, prediz o próximo token usando apenas a informação dos tokens anteriores.

O fluxo sequencial completo de uma sequência de texto dentro de uma LLM é: Texto Bruto  $\rightarrow$  Tokens  $\rightarrow$  Embeddings + Positional Enc.  $\rightarrow$  [Blocos Transformer]  $\times L \rightarrow$  RMSNorm  $\rightarrow$  Logits

Cada um dos  $L$  blocos Transformer é composto por duas subcamadas principais envoltas em conexões residuais:

$$\begin{aligned}x_{\text{attn}} &= x + \text{SelfAttention}(\text{RMSNorm}(x)) \\x_{\text{out}} &= x_{\text{attn}} + \text{MLP}(\text{RMSNorm}(x_{\text{attn}}))\end{aligned}$$

### 2.2 Self-Attention (Autoatenção)

#### 2.2.1 Conceito

O mecanismo de autoatenção permite que o modelo decida, de forma dinâmica, em quais partes da sequência ele deve "prestar atenção" para entender o token atual. Por exemplo, na frase: "O banco do rio estava úmido, então sentei longe do banco financeiro.", o modelo precisa saber que o primeiro "banco" se refere a uma estrutura natural, e o segundo "banco" se refere a uma instituição financeira. A atenção calcula essa associação contextualmente.

#### 2.2.2 Matemática: Fórmula Clássica

Dada uma matriz de entrada  $X \in \mathbb{R}^{T \times d_{\text{model}}}$ , nós projetamos  $X$  em três matrizes diferentes usando pesos lineares aprendidos:  $W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{model}} \times d_k}$  (onde  $d_k$  é a dimensão das chaves/valores):

$$Q = XW_Q \text{ (Queries)}, \quad K = XW_K \text{ (Keys)}, \quad V = XW_V \text{ (Values)}$$

- **Query (Q):** "O que o token atual está procurando?"
- **Key (K):** "O que o token tem a oferecer para os outros?"
- **Value (V):** "A informação real contida no token."

O cálculo da atenção ponderada é dado por:

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} + M \right) V$$

onde:

- $QK^T \in \mathbb{R}^{T \times T}$  calcula a pontuação de afinidade de cada par de tokens.
- $\sqrt{d_k}$  é um fator de escala para evitar logits excessivamente grandes que saturariam o gradiente da função softmax.
- $M$  é a máscara causal. Como o modelo é autoregressivo, ele não pode olhar para o futuro.  $M$  possui valores 0 para posições permitidas e  $-\infty$  para posições futuras.

### 2.2.3 Exemplo Numérico

Vamos calcular a atenção para um caso simplificado sem máscara causal, com  $T = 2$  tokens,  $d_k = 2$ :

$$Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad K = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad V = \begin{pmatrix} 2 & 3 \\ 1 & 4 \end{pmatrix}$$

#### 1. Passo 1: Multiplicação $QK^T$

$$K^T = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} \Rightarrow QK^T = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$$

#### 2. Passo 2: Escalonamento por $\sqrt{d_k} = \sqrt{2} \approx 1.414$

$$\text{Score} = \frac{1}{1.414} \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} \approx \begin{pmatrix} 0.707 & 0 \\ 0.707 & 0.707 \end{pmatrix}$$

#### 3. Passo 3: Aplicação da Softmax por linha

- Linha 1:  $[0.707, 0]$ .  $e^{0.707} \approx 2.028, e^0 = 1.0 \Rightarrow \text{Soma} = 3.028$ .  $P_{1,1} \approx 0.67, P_{1,2} \approx 0.33$ .
- Linha 2:  $[0.707, 0.707]$ . Como os valores são iguais, a softmax os divide igualmente:  $[0.50, 0.50]$ .

Temos a matriz de pesos de atenção:

$$A = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) = \begin{pmatrix} 0.67 & 0.33 \\ 0.50 & 0.50 \end{pmatrix}$$

#### 4. Passo 4: Multiplicação pelos Valores $V$

$$\text{Output} = A \cdot V = \begin{pmatrix} 0.67 & 0.33 \\ 0.50 & 0.50 \end{pmatrix} \begin{pmatrix} 2 & 3 \\ 1 & 4 \end{pmatrix} = \begin{pmatrix} 1.34 + 0.33 & 2.01 + 1.32 \\ 1.00 + 0.50 & 1.50 + 2.00 \end{pmatrix} = \begin{pmatrix} 1.67 & 3.33 \\ 1.50 & 3.50 \end{pmatrix}$$

O output final agora mesclou informações contextuais de ambos os tokens.

## 2.3 Multi-Head, MQA e Grouped-Query Attention (GQA)

A autoatenção padrão utiliza  $H$  cabeças independentes para capturar relações diferentes simultaneamente. No entanto, durante a inferência, armazenamos as chaves e valores anteriores em cache (KV Cache) para evitar recomputá-los a cada novo token gerado. Em sequências longas, esse KV Cache consome gigabytes de memória valiosa.

Para mitigar isso, surgiram variações de cabeças de atenção:

1. **Multi-Head Attention (MHA):** Cada cabeça de Query tem sua própria cabeça exclusiva de Key e Value. Consumo de memória do KV Cache é máximo.
2. **Multi-Query Attention (MQA):** Todas as cabeças de Query compartilham uma única cabeça de Key e Value física. Reduz drasticamente a memória do KV Cache, mas prejudica a capacidade de representação e qualidade do modelo.
3. **Grouped-Query Attention (GQA):** As cabeças de Query são divididas em grupos. Cada grupo compartilha uma única cabeça de Key e Value. Se tivermos  $H$  cabeças de Query e  $G$  grupos, temos  $G$  cabeças de Key e Value. A taxa de compressão do KV Cache é de  $H/G$ .

### Aplicação Prática nos Artigos: GQA nos Artigos Analisados

Quase todos os modelos modernos adotam GQA devido ao seu balanço perfeito entre qualidade linguística e eficiência do KV Cache:

- **Ministral 3** (3B, 8B, 14B) utiliza 32 cabeças de Query e 8 cabeças de Key/Value (um fator de compressão de 4:1).
- **LFM2** utiliza GQA em suas camadas globais (ex: no modelo de 2.6B, são 32 cabeças de Query para 8 de Key/Value).
- **Gemma 4** também utiliza GQA com o reuso de chaves como valores nas camadas globais de atenção para reduzir ainda mais o KV Cache footprint em até 37.5%.

## 2.4 Positional Encoding e Rotary Position Embeddings (RoPE)

### 2.4.1 Conceito

Como o mecanismo de atenção calcula a relação de afinidade por multiplicações matriciais puras (que são comutativas), ele é inerentemente invariante à ordem dos tokens. Se embaralharmos as palavras da frase, a matriz de autoatenção resultante será numericamente idêntica. Para que o modelo entenda a sintaxe e a ordem do texto, precisamos codificar a posição de cada token dentro do vetor.

### 2.4.2 Matemática do RoPE (Rotary Position Embedding)

O RoPE codifica a posição rotacionando os vetores de Query e Key em um espaço duplamente dimensional complexo. Em vez de somar um vetor de posição aos embeddings (como no Transformer original), o RoPE multiplica cada par de elementos adjacentes de Query ou Key por uma matriz de rotação que depende do índice da posição  $m$  do token.

Para um vetor bidimensional  $x = (x_1, x_2)^T$  na posição  $m$ , a rotação RoPE é:

$$R_{\theta, m} x = \begin{pmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

Onde  $\theta$  é a frequência angular base da dimensão do embedding. Para um vetor de dimensão completa  $d$ , agrupamos os elementos em pares de dimensões bidimensionais e rotacionamos cada par de forma independente.

A grande propriedade matemática do RoPE é que ele é relativo: o produto escalar entre uma Query rotacionada na posição  $m$  e uma Key rotacionada na posição  $n$  depende puramente da distância relativa  $m - n$ :

$$\langle R_{\theta,m}q, R_{\theta,n}k \rangle = q^T R_{\theta,n-m}k$$

#### Aplicação Prática nos Artigos: Extensões e RoPE Bidimensional

- **Gemma 4 12B** (modelo encoder-free) e o **LFM2-VL** inovam ao estender o RoPE para a modalidade visual: utilizam RoPE 2D. Em vez de rotacionar chaves de acordo com um índice linear unidimensional de texto, o RoPE 2D rotaciona as representações espaciais dos patches de imagem nas coordenadas  $(x, y)$ , permitindo que o modelo preserve relações de vizinhança bidimensional (esquerda/direita, cima/baixo) diretamente no espaço latente.
- Para estender o contexto além do limite de treino original (ex: de 4k para 32k ou 256k), modelos como o **phi-4** e o **LFM2** alteram dinamicamente a base da frequência do RoPE (de 10.000 para 250.000), uma técnica conhecida como escala de frequência RoPE (p-RoPE), evitando que o modelo perca a estabilidade dos gradientes ao ver posições muito grandes durante a inferência.

## 2.5 RMSNorm e SwiGLU

Modernas SLMs utilizam componentes mais eficientes que substituem os tradicionais módulos do Transformer clássico.

### 2.5.1 RMSNorm (Root Mean Square Layer Normalization)

Ao contrário do tradicional LayerNorm que calcula a média e a variância dos elementos para normalizar uma camada, o RMSNorm assume que a média é próxima de zero e normaliza o vetor puramente pela raiz de sua média quadrática (Root Mean Square):

$$y = \frac{x}{\text{RMS}(x)} \odot \gamma, \text{ onde } \text{RMS}(x) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}$$

Onde  $\gamma \in \mathbb{R}^d$  é um vetor de escala aprendido e  $\epsilon$  é uma pequena constante para evitar divisão por zero. O RMSNorm alcança as mesmas propriedades de estabilidade de treinamento que o LayerNorm original, mas com uma redução de 7% a 10% no custo computacional, pois evita calcular e subtrair a média do vetor.

### 2.5.2 SwiGLU (Swish Gated Linear Unit)

A subcamada MLP (Feed-Forward) tradicional usava ativações ReLU ou GeLU. Modelos modernos utilizam o bloco SwiGLU, um mecanismo com portas (Gated) que se provou estatisticamente superior:

$$\text{SwiGLU}(x) = (\text{Swish}(xW_{\text{gate}}) \otimes (xW_{\text{up}}))W_{\text{down}}$$

Onde:

- $\text{Swish}(x) = x \cdot \sigma(\beta x)$  (geralmente com  $\beta = 1$ , o que é equivalente à função SiLU).

- $W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$  são pesos lineares aprendidos da rede MLP.
- $\otimes$  representa a multiplicação elemento a elemento (Hadamard product).

O SwiGLU permite que a rede aprenda uma função de porta (gate) dinâmica que filtra e modula os sinais de ativação de forma não linear, aumentando consideravelmente o poder de expressão das representações latentes do modelo sem sobrecarregar o número de parâmetros ativos.

## 2.6 Exercícios e Resumos

### Erro Conceitual Comum: Confundir GQA com MQA

Um erro comum é achar que GQA e MQA são o mesmo algoritmo. No MQA, todas as cabeças de Query do modelo inteiro dividem uma única cabeça de Key e Value. No GQA, as Queries são organizadas em grupos (ex: grupos de 4 ou 8 Queries), e cada grupo compartilha uma cabeça exclusiva de Key e Value para aquele grupo. GQA é uma generalização do MQA e do MHA, permitindo configurar a granularidade ideal de acordo com o hardware.

### 2.6.1 Resumo do Capítulo

- O Transformer decoder-only processa tokens contextualmente usando atenção causal restrita ao passado.
- A autoatenção projeta vetores de entrada em Queries, Keys e Values para mapear afinidades contextuais dinâmicas.
- O GQA reduz a pegada do KV Cache compartilhando cabeças de Key e Value entre grupos de cabeças de Query.
- O RoPE aplica uma matriz de rotação para codificar posições relativas na atenção, podendo ser estendido para 2D para processar imagens e áudio de forma espacial nativa.
- RMSNorm remove cálculos de médias redundantes, e SwiGLU introduz MLPs com portas não lineares altamente eficientes.

### 2.6.2 Exercícios

**Exercício 1:** Explique matematicamente por que o fator de escala  $\sqrt{d_k}$  na fórmula da atenção é importante. O que aconteceria com os gradientes se ele fosse omitido?

**Solução:** Considere que os componentes dos vetores de chaves K e queries Q são variáveis aleatórias independentes com média 0 e variância 1. O produto escalar  $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$  terá média 0, mas sua variância será exatamente  $d_k$  (a soma de  $d_k$  termos de variância 1). Para dimensões grandes (ex:  $d_k = 128$ ), o valor do produto escalar pode ser extremamente alto ou baixo. Se passarmos esses valores sem escala para a função exponencial  $\text{softmax}(z_i) = e^{z_i} / \sum e^{z_j}$ , o maior logit dominará completamente a distribuição, empurrando as probabilidades para perto de 1.0 e 0.0. Na região de saturação da Softmax, suas derivadas em relação aos parâmetros de entrada tornam-se quase zero (gradiente nulo), o que interrompe o aprendizado por backpropagation. Ao escalonar o produto escalar por  $1/\sqrt{d_k}$ , a variância é restaurada para exatamente 1.0, mantendo a função softmax em sua zona de maior sensibilidade e fluxo de gradientes saudável.

## Capítulo 3

# Funcionamento Extremo, Memória e Ciclo de Pré-treinamento

Entendida a arquitetura Transformer, precisamos examinar como ela é otimizada em larga escala sob restrições físicas de hardware, desde a alocação de memória de precisão numérica até os pipelines massivos de pré-treinamento.

### 3.1 Arquitetura Completa de uma LLM: O Fluxo de Dados

Podemos agora mapear geometricamente o fluxo de dados em um único diagrama de processamento que detalha a inferência autoregressiva:

1. **Input:** O usuário escreve o texto "O gato bebeu...".
2. **Tokenização:** O texto é mapeado para índices discreteados: [1420, 8931, 23512].
3. **Embedding Lookup:** Cada índice de token é transformado em um vetor denso  $x \in \mathbb{R}^{d_{\text{model}}}$  de dimensão oculta.
4. **Camadas Transformer:** A sequência de vetores passa por  $L$  camadas consecutivas de autoatenção causal (GQA), posições rotacionadas (RoPE) e normalizações (RMSNorm) com camadas MLP (SwiGLU).
5. **Projection (Unembedding):** O vetor latente final da última camada para o último token é projetado na matriz  $W_{\text{out}}$  para produzir um vetor de logits de tamanho  $V$ .
6. **Softmax:** Os logits são convertidos em probabilidades.
7. **Amostragem (Sampling):** Um token é escolhido (ex: "leite") e anexado ao final do input para a próxima passada causal.

### 3.2 Parâmetros, Precisão e Pegada de Memória

#### 3.2.1 Diferenciando Parâmetros Totais de Parâmetros Ativados

Nas arquiteturas Mixture-of-Experts (MoE), nem todos os neurônios da rede são computados para cada token processado. Enquanto o modelo armazena uma matriz massiva de especialistas em disco, apenas um subconjunto de especialistas ativos (Top-k) é selecionado por um roteador para processar aquele token específico. Assim:

- **Parâmetros Totais ( $N_{\text{total}}$ ):** É a soma de todos os pesos contidos em disco. Define quanta RAM/VRAM de armazenamento é necessária para carregar o modelo.

- **Parâmetros Ativos ( $N_{\text{active}}$ ):** É a fração de parâmetros que efetivamente realizam computações matemáticas para um token. Define o custo computacional (FLOPs por token) e a latência de execução.

### 3.2.2 Precisão Numérica e Memória de Armazenamento

Os pesos de um modelo são armazenados em formatos com diferentes resoluções de bits. A pegada de memória RAM para guardar apenas os pesos é calculada multiplicando o número de parâmetros pelo número de bytes por parâmetro:

- **FP32** (Precisão de 32 bits de ponto flutuante): 4 bytes por parâmetro.
- **FP16 / BF16** (Precisão de 16 bits): 2 bytes por parâmetro.
- **INT8** (Inteiros de 8 bits): 1 byte por parâmetro.
- **INT4** (Inteiros de 4 bits): 0.5 bytes por parâmetro.

#### Exemplo Prático: Cálculo de Pegada de Memória para Modelo 7B

Suponha que desejamos executar uma SLM de 7 bilhões de parâmetros (ex: LFM2-VL-3B ou similar de 7B) no celular. Quanta RAM precisamos?

- Em FP16:  $7 \times 10^9 \text{ params} \times 2 \text{ bytes} = 14 \text{ GB}$
- Em INT8:  $7 \times 10^9 \text{ params} \times 1 \text{ byte} = 7 \text{ GB}$
- Em INT4:  $7 \times 10^9 \text{ params} \times 0.5 \text{ bytes} = 3.5 \text{ GB}$

Isso demonstra que a quantização (reduzir a precisão numérica para INT4) é estritamente necessária para que modelos de médio porte caibam em dispositivos do consumidor (edge) com RAM restrita.

#### Aplicação Prática nos Artigos: Quantização de Ultra Baixa Pegada

Segundo o relatório da Gemma 4, as otimizações de Quantization-Aware Training (QAT) em diferentes formatos permitem economias impressionantes de footprint (Tabela 3 do artigo original):

- O modelo **Gemma 4 E2B** bruto (FP16) ocupa 4.6 GB de memória. Em versão móvel quantizada, essa pegada despenca para 0.8 GB.
- O modelo **12B** original ocupa 24.0 GB de VRAM, mas cai para 7.65 GB após quantização Q4\_0.
- Essas quantizações são acompanhadas de um *scalar scale* para conter a amplitude dinâmica das ativações e evitar erros de underflow em representações de poucos bits.

## 3.3 Ciclo de Pré-treinamento

O pré-treinamento é a fase mais cara e demorada do desenvolvimento de uma LLM. O modelo lê trilhões de tokens sem anotações humanas e aprende a completar o próximo token a partir do passado em um regime puramente auto-supervisionado.

### 3.3.1 Loss de Entropia Cruzada Causal

O modelo é treinado minimizando a perda de entropia cruzada em cada passo de predição temporal. Para um vocabulário  $V$ , dados os logits previstos pela última camada  $z \in \mathbb{R}^V$ , calculamos as probabilidades:

$$P(y = i|x) = \frac{e^{z_i}}{\sum_{j=1}^V e^{z_j}}$$

A perda para o token correto cujo índice é  $c$  é simplesmente a log-probabilidade negativa:

$$L = -\log P(y = c|x) = \log \left( \sum_{j=1}^V e^{z_j} \right) - z_c$$

### 3.3.2 AdamW (Decoupled Weight Decay Adam)

Para atualizar os pesos com segurança e velocidade, o otimizador AdamW rastreia as estimativas do primeiro e segundo momentos dos gradientes para regular a taxa de aprendizado para cada parâmetro individualmente. As equações formais de atualização a cada passo  $t$  para os pesos  $\theta$  são:

$$\begin{aligned} g_t &= \nabla_{\theta} L_t \\ m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (1^{\text{o}} \text{ momento: Média Móvel dos Gradientes}) \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (2^{\text{o}} \text{ momento: Média Móvel dos Quadrados}) \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (\text{Correções de Viés}) \\ \theta_t &= \theta_{t-1} - \eta \cdot \left( \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right) \end{aligned}$$

Onde  $\eta$  é a learning rate (taxa de aprendizado),  $\lambda$  é o fator de decaimento de peso (weight decay) e  $\epsilon$  é um valor infinitesimal para estabilidade. No Adam tradicional, o decaimento de peso era aplicado diretamente no gradiente modificado pela escala de variância  $\sqrt{v_t}$ . O AdamW corrige esse problema desacoplando a regularização do decaimento de peso de forma direta sobre os pesos físicos  $\theta$ , mantendo o decaimento idêntico ao do algoritmo SGD clássico, o que melhora consideravelmente a generalização de representações semânticas profundas em LLMs.

## 3.4 Exercícios e Resumos

### Erro Conceitual Comum: Confundir Parâmetros Totais com Custo de Inferência em MoE

Achar que um modelo Mixture-of-Experts com 8.3B de parâmetros (como o LFM2-8B-A1B) terá a mesma latência e custo computacional de inferência que um modelo denso de 8.3B de parâmetros. Na inferência, o custo é ditado pelos parâmetros ativos. Como o LFM2-8B-A1B ativa apenas 1.5B de parâmetros por token (com Top-k=4 de 32 especialistas), sua latência de execução é idêntica à de um modelo denso super-rápido de 1.5B, enquanto mantém a precisão de um modelo muito maior (3B a 4B).

### 3.4.1 Resumo do Capítulo

- O pipeline de inferência de uma LLM decodifica tokens sequencialmente, convertendo embeddings em logits e probabilidade softmax.

- A pegada de memória do modelo quantizado cai de bfloat16 para INT4 em até 4 vezes, viabilizando edge-computing e on-device AI.
- A perda de entropia cruzada penaliza o modelo proporcionalmente ao erro de probabilidade na predição do próximo token causal.
- O AdamW desacopla o weight decay do termo do segundo momento gradiente, preservando a regularização ideal da rede.

### 3.4.2 Exercícios

**Exercício 1:** Um modelo de linguagem causal possui uma dimensão de embedding  $d_{\text{model}} = 2048$ ,  $N_{\text{layers}} = 16$ , com  $H = 16$  cabeças de Query e  $G = 4$  cabeças de Key/Value (GQA), em precisão de 16-bits (FP16, 2 bytes). Calcule o tamanho em MB do KV Cache acumulado para uma sequência de comprimento  $T_{\text{context}} = 8192$  com tamanho de lote (batch size)  $B = 1$ .

**Solução:** O KV Cache armazena as chaves e valores anteriores em todas as camadas. A dimensão de cada cabeça (head size) é  $d_{\text{head}} = d_{\text{model}}/H = 2048/16 = 128$ . Como temos GQA, o número de chaves e valores armazenados por camada é ditado pelas  $G$  cabeças de Key/Value, e não pelas  $H$  cabeças de Query. Para cada camada, armazenamos para cada token:

- Vetor chave:  $G \times d_{\text{head}} = 4 \times 128 = 512$  elementos.
- Vetor valor:  $G \times d_{\text{head}} = 4 \times 128 = 512$  elementos.
- Total por camada por token:  $512 + 512 = 1024$  elementos.

O total de elementos armazenados em todo o modelo para  $T_{\text{context}} = 8192$  e  $B = 1$  é:

$$\begin{aligned}\text{Total Elementos} &= N_{\text{layers}} \times T_{\text{context}} \times B \times (\text{Elementos por token por camada}) \\ &= 16 \times 8192 \times 1 \times 1024 = 1.342.177.280 \text{ elementos}\end{aligned}$$

Como cada elemento em FP16 consome 2 bytes, a memória total consumida em bytes é:

$$1.342.177.280 \times 2 = 2.684.354.560 \text{ bytes} \approx 2.68 \text{ GB (ou 2560 MB)}$$

Note que se tivéssemos utilizado o Multi-Head Attention (MHA) clássico sem agrupamento GQA, o cálculo usaria  $H = 16$  em vez de  $G = 4$ , gerando um cache 4 vezes maior de aproximadamente 10.72 GB, impossibilitando a execução em dispositivos móveis padrão.

## Capítulo 4

# Pós-treinamento, Alinhamento de Preferências e Raciocínio

Um modelo pré-treinado em trilhões de tokens é apenas um completador de texto extremamente sofisticado; se você lhe fizer uma pergunta, ele pode simplesmente inventar uma resposta contínua ou repetir a pergunta. Para torná-lo útil como assistente prático, seguro e capaz de resolver problemas abstratos, aplicamos o Pós-treinamento.

### 4.1 Pré-treinamento vs. Fine-tuning vs. Pós-treinamento

Diferenciamos esses três processos pelas seguintes características fundamentais:

| Característica | Pré-treinamento               | Supervised Fine-Tuning (SFT)        | Alinhamento (RLHF/DPO)                   |
|----------------|-------------------------------|-------------------------------------|------------------------------------------|
| Objetivo       | Aprender linguagem e fatos    | Seguir instruções e formatos        | Comportamento, tom e preferência         |
| Dados          | Trilhões de tokens (brutos)   | Milhares de exemplos de diálogos    | Pares de respostas (preferida/rejeitada) |
| Computação     | Extremamente alta (SOTA pods) | Baixa a média (alguns nós GPU)      | Média a alta (alguns nós GPU)            |
| Loss Usada     | Entropia Cruzada Causal       | Entropia Cruzada (apenas no output) | Loss DPO / Perdas de Reforço (RL)        |

### 4.2 Instruction Tuning e SFT

O Supervised Fine-Tuning (SFT) treina o modelo para responder no formato de assistente de diálogo usando conjuntos de dados com conversas curadas e polidas. O modelo aprende a usar modelos estruturados de mensagens para isolar o papel do usuário do papel do sistema (ex: ChatML):

```
<| im_start | > system
Voce e um assistente util especialista em matematica . <| im_end | >
<| im_start | > user
Quanto e 2 + 2? <| im_end | >
<| im_start | > assistant
O resultado e 4. <| im_end | >
```

Durante o SFT, nós mascaramos a perda de entropia cruzada para que o modelo seja treinado e atualizado apenas sobre os tokens correspondentes às respostas do assistente, evitando perturbar a base de conhecimentos do pré-treino com as instruções de entrada.

### 4.3 RL Aplicado a LLMs: PPO, DPO e GRPO

Para alinhar as respostas do modelo de forma a serem úteis, verdadeiras e seguras, utilizamos o aprendizado por reforço.

1. **PPO (Proximal Policy Optimization):** Método clássico que exige treinar quatro modelos diferentes em paralelo (a política de atuação, um modelo de valor Critic, um modelo de recompensa Reward e uma política de referência). É extremamente pesado, instável para treinar e consome imensa VRAM.
2. **DPO (Direct Preference Optimization):** Uma inovação revolucionária que dispensa o modelo de recompensa e o critic. Ela formula a perda diretamente em cima da probabilidade de o modelo gerar a resposta "preferida" ( $y_w$ ) em vez de uma resposta "rejeitada" ( $y_l$ ). A perda DPO é calculada por:

$$L_{DPO}(\theta; \pi_{\text{ref}}) = -E_{(x, y_w, y_l) \sim D} \left[ \log \sigma \left( \beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]$$

Onde  $\pi_{\text{ref}}$  é a política congelada de referência (para evitar que o modelo sofra colapso catastrófico de linguagem) e  $\sigma$  é a função sigmoide.

3. **GRPO (Group Relative Policy Optimization):** Otimização criada para modelos de raciocínio lógico profundo. Em vez de utilizar um modelo Critic separado para estimar o valor de base de um estado, o GRPO gera um grupo de  $G$  respostas alternativas para o mesmo prompt, pontua todas elas usando uma função de recompensa (humana ou programática) e calcula a vantagem de cada resposta em relação à média do grupo:

$$A_i = \frac{r_i - \text{mean}(R)}{\text{std}(R)}$$

Isso reduz a pegada de VRAM em mais de 30%, permitindo o escalonamento massivo de treino de raciocínio em clusters menores.

## 4.4 Chain-of-Thought (CoT) vs. Raciocínio (Reasoning)

Diferenciar o mero formato de saída do processo de exploração cognitiva em LLMs é de suma relevância técnica:

- **Chain-of-Thought (CoT):** É um formato estruturado de texto onde o modelo simplesmente descreve o passo a passo lógico antes de fornecer a resposta final (ex: "Vamos pensar passo a passo..."). Pode ser emulado via SFT estático ou prompts direcionados.
- **Reasoning (Raciocínio):** Refere-se a modelos treinados ativamente via Reinforcement Learning (como GRPO) para realizar exploração de caminhos mentais, auto-correção em tempo real e backtracking (voltar atrás ao perceber erros). O modelo adquire um modo de raciocínio intrínseco (como no Gemma 4 Thinking Mode ou Ministral 3 Reasoning), permitindo escalar a qualidade das respostas proporcionalmente ao tempo de computação usado na inferência (test-time compute scaling).

## 4.5 Exercícios e Resumos

### Erro Conceitual Comum: Achar que CoT e Reasoning são Sinônimos

Achar que qualquer modelo que escreve uma resposta longa com "Pensando..." ou passo a passo possui capacidades reais de raciocínio. Um modelo simples pode apenas reproduzir o estilo estilístico de escrita CoT sem ter a habilidade real de avaliar logicamente suas hipóteses e corrigir-se caso cometa um erro matemático no início da sequência. O raciocínio de verdade requer treinamento ativo por reforço em cima de métricas estruturadas de verificação de soluções (como compiladores de código ou juízes de equações matemáticas).

### 4.5.1 Resumo do Capítulo

- O SFT orienta o modelo a formatar saídas no padrão assistente sem reescrever conhecimentos do pré-treino.
- DPO descarta loops PPO complexos, alinhando de forma direta as probabilidades em pares preferidos/rejeitados.
- GRPO elimina o modelo Critic em tarefas de raciocínio, calculando vantagens estatísticas relativas a grupos de candidatos.
- Raciocínio complexo é habilitado por incentivos por reforço (RL) que encorajam autocorreção ativa, e não mero preenchimento estilístico de CoT.

### 4.5.2 Exercícios

**Exercício 1:** Considere a perda de um par DPO. Se as probabilidades relativas de referência de duas respostas candidatas  $y_w$  e  $y_l$  são iguais:  $\pi_{\text{ref}}(y_w|x) = 0.2$  e  $\pi_{\text{ref}}(y_l|x) = 0.2$ , e sob os novos pesos  $\theta$  temos  $\pi_{\theta}(y_w|x) = 0.5$  e  $\pi_{\theta}(y_l|x) = 0.1$ . Dado  $\beta = 1.0$ , calcule o valor que entra na função sigmoide  $\sigma$  na perda DPO.

**Solução:** O argumento dentro da sigmoide é:

$$\begin{aligned}\Delta &= \beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} = 1.0 \times \log \left( \frac{0.5}{0.2} \right) - 1.0 \times \log \left( \frac{0.1}{0.2} \right) \\ &= \log(2.5) - \log(0.5) = 0.916 - (-0.693) = 1.609\end{aligned}$$

Como  $\Delta = 1.609 > 0$ , o valor sigmoide resultante será  $\sigma(1.609) \approx 0.833$ , o que fará a perda DPO ser baixa ( $-\log(0.833) \approx 0.183$ ), mostrando que os parâmetros se alinharam perfeitamente para favorecer a resposta correta em relação à errada.

## Capítulo 5

# Análise Crítica e Comparativa dos Artigos Científicos

### 5.1 Gemma 4: Fronteira de Multimodalidade e Raciocínio (DeepMind)

#### 5.1.1 Problema e Motivação

Dispositivos móveis e IoT demandam modelos capazes de processar nativamente áudio, texto e visão simultaneamente. Soluções anteriores utilizavam encoders externos gigantescos acoplados aos LLMs, gerando fragmentação massiva de memória e latência severa.

#### 5.1.2 Arquitetura e Inovações Técnicas

- **Thinking Mode:** O modelo gera traços lógicos profundos ocultos antes de fornecer respostas para matemática e codificação.
- **Long Context Footprint:** Usa p-RoPE e uma razão de atenção de 5:1 (local sliding window para global self-attention), combinando o reuso de chaves como valores nas camadas globais, cortando o KV cache footprint em 37.5% (32k de contexto).
- **Encoder-Free (Gemma 4 12B):** Remove totalmente os encoders externos dedicados de imagem (550M) e áudio USM (305M). Em vez disso, projeta patches de imagem RGB  $48 \times 48 \times 3$  com coordenadas espaciais 2D via RoPE e chunks de áudio bruto de 40ms diretamente no espaço de embeddings do LLM, economizando parâmetros e evitando fragmentação do hardware.

#### 5.1.3 Resultados e Benchmarks

Gemma 4 31B obteve Elo de 1451 no Arena Text Leaderboard, estabelecendo-se como o modelo denso aberto de maior pontuação, superando o Kimi K2.5 Thinking de tamanho muito superior (Tabela 4 do artigo).

### 5.2 LFM2: Liquid Foundation Models de Extrema Velocidade (Liquid AI)

#### 5.2.1 Problema e Motivação

Modelos baseados puramente em atenção sofrem com a latência de preenchimento e custos exponenciais de memória em comprimentos longos de contexto no edge (CPUs móveis).

### 5.2.2 Arquitetura: Minimal Hybrid

O LFM2 descarta abordagens puras de atenção e evita o uso complexo de SSMS lineares ou Mamba ao introduzir uma descoberta por busca automática de arquitetura (hardware-in-the-loop):

- Combina uma ampla maioria de gated short convolutions em 1D (convoluções locais leves e rápidas) com uma pequena minoria estratégica de camadas de atenção GQA globais.
- Essa configuração alcança desempenho idêntico ao de modelos complexos, reduzindo drasticamente o consumo de memória RAM do KV cache.

### 5.2.3 Pre-training: Decoupled Top-K Distillation

A fim de destilar o conhecimento de um professor maior (LFM1-7B) para o estudante compacto sem problemas de desajuste de suporte estatístico, o LFM2 introduz uma perda de destilação desacoplada de temperatura temperada (decoupled Top-K knowledge distillation), garantindo estabilização de convergência ao longo de 10T a 12T tokens de pré-treinamento.

### 5.2.4 Modelos Especializados e Resultados

- **LFM2-VL e LFM2-Audio:** O primeiro adota tiling dinâmico para visual de alta fidelidade e ajuste de tokens por imagem; o último expõe caminhos de áudio contínuo independentes para resposta instantânea em voz.
- **Inference Performance:** No Snapdragon 8 Elite (Galaxy S25), o LFM2-1.2B superou o Llama-3.2-1B em mais de 1.5x na velocidade de prefill de tokens (Tabela 2 do artigo).

## 5.3 Ministral 3: Cascade Distillation e Análise de Professores (Mistral AI)

### 5.3.1 Problema e Motivação

Treinar pequenos modelos do zero (from scratch) exige dezenas de trilhões de tokens e enorme custo de processamento financeiro. A Mistral AI busca reaproveitar o conhecimento consolidado de grandes checkpoints.

### 5.3.2 Arquitetura e Cascade Distillation

O Ministral 3B, 8B e 14B são gerados por um pipeline iterativo apelidado de Cascade Distillation:

1. Poda-se (Pruning) o modelo pai Mistral Small 3.1 24B para inicializar o filho de 14B.
2. Realiza-se destilação contínua de logits causal em janelas de contexto curto com o pai como professor.
3. O modelo resultante é podado novamente para gerar o inicializador de 8B, repetindo o processo até chegar ao de 3B.

### 5.3.3 Ablações Técnicas Cruciais do Artigo

Os pesquisadores da Mistral estabeleceram três descobertas empíricas fundamentais sobre o uso de professores em destilação (Seção 5.1 do artigo):

- **Capacidade do Professor:** No pré-treinamento, destilar do Mistral Small 3.1 (24B) gerou resultados melhores do que destilar do Mistral Medium 3, que é muito maior e inteligente. Há um limite físico de aprendizado para o estudante, de modo que professores excessivamente complexos criam divergências intransponíveis no espaço latente.
- **Pós-treinamento Beneficia do Maior Professor:** Em contraste direto com o pré-treinamento, a fase de pós-treinamento SFT de modelos menores se beneficia diretamente de professores maiores e mais inteligentes (como Mistral Medium).
- **Professores Preference-Tuned:** Ensinar o estudante usando logs de um professor treinado por reforço de preferências humanas (DPO/RLHF) produz checkpoints consideravelmente melhores do que usar logs de professores treinados apenas via SFT simples.

## 5.4 Phi-4: Supremacia Lógica via Dados Sintéticos Criteriosos (Microsoft)

### 5.4.1 Problema e Motivação

Pequenos modelos tipicamente falham em tarefas científicas pesadas (matemática, biologia, programação) devido à escassez de dados acadêmicos estruturados de alta qualidade e à presença de ruídos nos corpora públicos da internet.

### 5.4.2 Estratégia de Dados de Pretraining

O phi-4 usa dados sintéticos como o bloco central do seu pré-treinamento (40% do mixture total de tokens, Tabela 5). Esses dados são gerados via workflows rigorosos:

- **Multi-agent prompting e self-revision:** Diferentes personas de LLMs revisam e validam a correção de equações geradas para garantir que não ocorra a propagação de erros sistemáticos nos dados de treino.

### 5.4.3 Post-Training: Pivotal Token Search (PTS) DPO

Para além do SFT refinado, o phi-4 introduz uma metodologia inovadora para criar pares de preferência para o DPO: o Pivotal Token Search (PTS). Em tarefas lógicas onde a resposta pode ser validada (ex: programação, equações), o algoritmo PTS rastreia o momento exato em que o modelo decodificou um token que determinou o sucesso ou o fracasso de resolver aquela equação. Ele isola esse token crítico (pivotal) e cria uma amostra preferida (com o token que leva à vitória) e uma rejeitada (com o token alternativo que desvia para o erro), gerando dados DPO extremamente densos em sinal útil de treino.

### 5.4.4 Métricas, Resultados e Limitações

- **AMC Benchmark (November 2024):** Testado em exames inéditos pós-coleta de dados, o phi-4 (14B) obteve nota 91.8, superando largamente o GPT-4o e o Gemini Pro 1.5, comprovando robustez lógica real e total ausência de contaminação de dados.
- **Limitações:** Menos proficiente em seguir regras de formatação minuciosas (ex: tabelas estritas, bullet points longos) pois sua base sintética priorizou Q&A lógico profundo em vez de alinhamento textual de estilo (IFEval modesto de 63.0 vs 78.7 do Qwen-2.5-14B). Sofre de problemas de verbosidade excessiva por usar muito CoT.

| Modelo      | Tamanho (Active/Total)     | Contexto   | Inovação de Treino           | Inovação Arquitetural      |
|-------------|----------------------------|------------|------------------------------|----------------------------|
| Gemma 4     | 2.3B a 31B (MoE 3.8/26B)   | 32k a 128k | QAT & spec. decoding         | Encoder-free 12B, Thinking |
| LFM2        | 350M a 2.6B (MoE 1.5/8.3B) | 32k        | Decoupled Top-K Distill.     | Gated Convolutions + GQA   |
| Ministral 3 | 3B, 8B, 14B (Denses)       | 256k       | Cascade Distillation         | Tied Emb. (3B), pixtral V  |
| phi-4       | 14B (Dense)                | 16k        | Synthetic pre-train, PTS DPO | Tiktoken Tokenizer (100k)  |

## 5.5 Tabela de Síntese Comparativa

## 5.6 Exercícios e Resumos

### Erro Conceitual Comum: Achar que phi-4 e Gemma 4 Distilam puramente o GPT-4

Achar que estes modelos são meros alunos preguiçosos que copiam o GPT-4 e nunca poderão ultrapassá-lo. O phi-4 ultrapassa diretamente o GPT-4o em avaliações STEM cruciais como o GPQA Diamond (56.1% vs 50.6%) e AMC Math, e isso acontece porque seus pipelines de autoria de dados sintéticos aplicam regras de validação por restrições simbólicas e programação de compiladores que geram raciocínios e provas lógicas de alta consistência matemática que nem mesmo o GPT-4o original conseguia formular rotineiramente de forma livre.

### 5.6.1 Resumo do Capítulo

- O Gemma 4 inova com seu modo de pensamento intrínseco e sua versão 12B livre de encoders de áudio/visão.
- O LFM2 rompe o gargalo computacional do preenchimento da atenção ao usar convoluções locais com camadas esparsas de GQA.
- O Ministral 3 detalha que a destilação de pre-training requer professores de tamanho comedido (Mistral Small) para evitar incompatibilidades de representação, enquanto a fase SFT se beneficia diretamente de modelos de escala fechada máxima (Mistral Medium).
- O phi-4 foca na pureza de dados sintéticos e no refinamento cirúrgico de gradientes de preferência via Pivotal Token Search DPO.

### 5.6.2 Exercícios

**Exercício 1:** Baseado nas descobertas do artigo "Ministral 3", descreva o fenômeno do "capacity gap" (limite de aprendizado) na destilação de modelos de linguagem pré-treinados. Quais as suas implicações práticas para engenheiros e cientistas de dados ao projetarem novos SLMs?

**Solução:** O "capacity gap" na destilação de pre-training descreve a perda de desempenho observada quando um modelo estudante de tamanho muito pequeno tenta mimetizar os logits ou ativações internas de um professor excessivamente poderoso e grande. Praticamente, a Mistral AI demonstrou que destilar o Ministral 14B do Mistral Small 3.1 (24B) produziu resultados consistentemente superiores em comparação com destilá-lo do muito mais inteligente Mistral Medium 3. Isso ocorre porque a distribuição de probabilidades no espaço de embeddings e logits de um modelo gigantesco possui uma entropia, estrutura dimensional e riqueza abstrata que o estudante pequeno de baixa capacidade dimensional simplesmente não possui parâmetros suficientes para internalizar. Tentar aproximar essas representações gera perdas de estabilidade e sub-otimização geral (underfitting). Para engenheiros práticos, a implicação é direta:

1. Ao realizar destilação contínua durante o pré-treinamento, o professor escolhido deve estar na escala dimensional imediatamente superior à do estudante (ex: treinar um modelo de 3B com um de 8B ou 14B), e não no nível de frontier de 400B+.
2. Deixe para usar o professor de fronteira máxima (ex: GPT-4o, Mistral Medium) exclusivamente nas etapas de pós-treinamento (SFT, geração de instruções), onde o modelo estudante precisa apenas aprender estilos superficiais de conversação e caminhos de formatação textual e não as estruturas geométricas internas profundas do pre-training.